

Image Segmentation

1. Introduction

Semantic segmentation assigns a class label to every pixel, Which is important for applications such as autonomous driving and scene understanding. This project performs 5-class segmentation (background, person, car, book, airplane) on a 350-image ADE20K subset, comparing a U-Net against a DeepLabV3-ResNet50 model pretrained on COCO and fine-tuned on our data.

Zhou et al. (2017) introduced ADE20K as a densely annotated scene-parsing benchmark and established the per-class/mean-IoU evaluation protocol this project follows. Ronneberger et al. (2015) proposed skip connections to keep important spatial details when reducing image size, and we use this design for our model training. Chen et al. (2017) introduced DeepLabV3, using atrous convolutions and Atrous Spatial Pyramid Pooling to capture multi-scale context without losing image resolution, we use its COCO-pretrained ResNet-50 variant as our transfer-learning comparison. Together these motivate our expectation, tested below, that pretraining should help most where our own data is scarcest.

2. Data Description and Exploratory Data Analysis

The dataset comprises 350 training, 350 validation, and 30 unlabeled test images, COCO-annotated across 100 ADE20K categories, of which we use 4. Class frequency is severely imbalanced: person appears in 233 training images (1084 instances), car in 148 (689), book in 51 (155), and airplane in only 5 (13 instances).

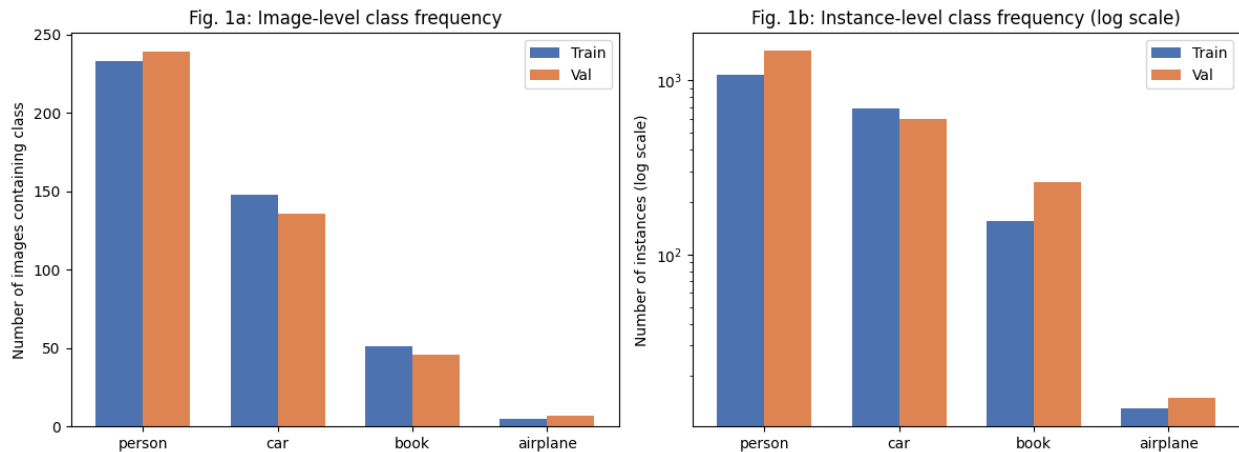


Figure 1: Image- and instance-level frequency per target class

At pixel level the imbalance is starker still: background occupies 93.5% of training pixels, versus 3.8% person, 2.3% car, 0.29% book, and 0.12% airplane.

Fig. 4: Pixel share per class (train) - treemap view

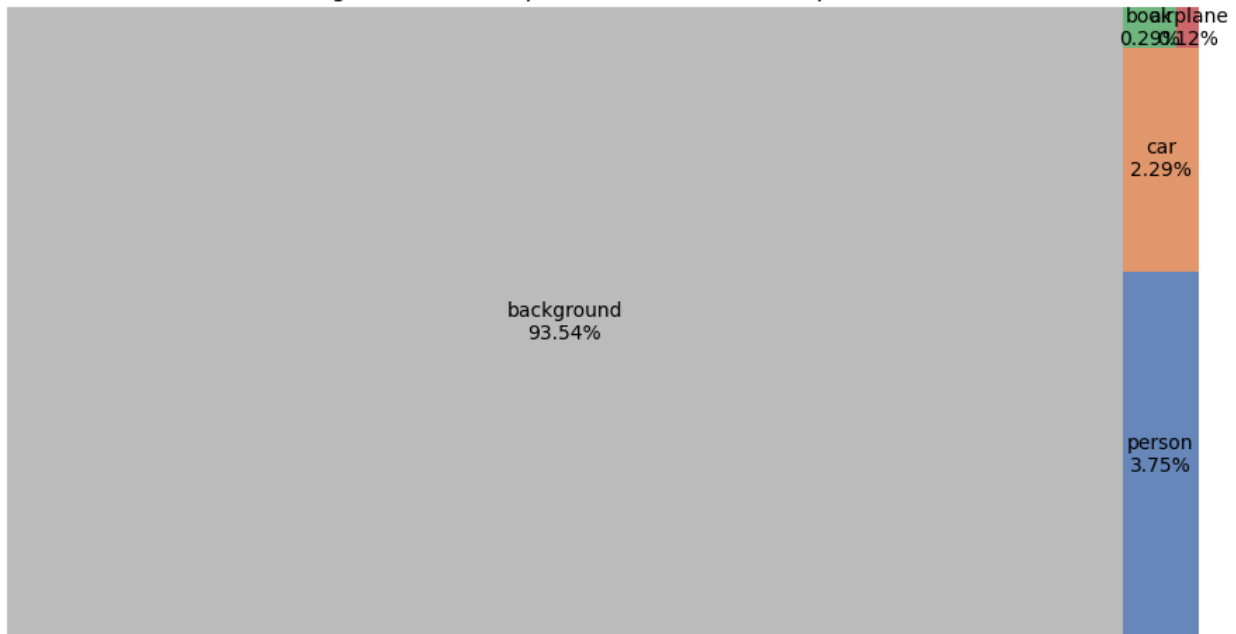


Figure 2: Treemap of pixel-level class share.

Instance-size analysis (Fig. 3) shows book objects are also disproportionately small, compounding rarity with a boundary-precision problem: a mislabeled boundary is a far larger relative error for a small object than a large one.

Fig. 4: Distribution of instance size per class (train, log scale)

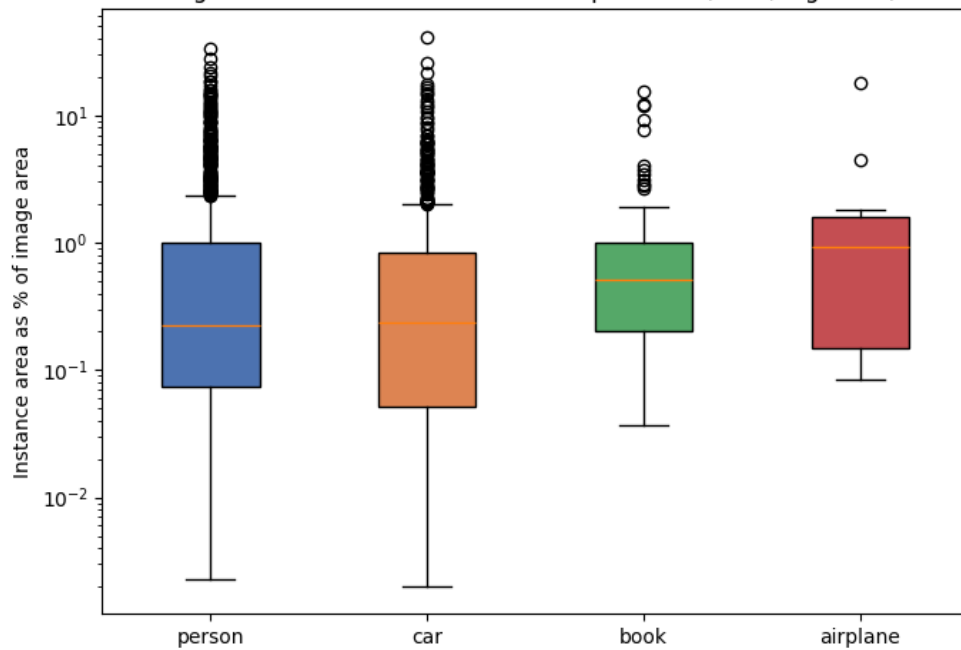


Figure 3: Instance area (% of image) per class, log scale

Co-occurrence analysis (Fig. 4) shows person and car frequently appear together, while airplane rarely cooccur with other targets, limiting how much any one evaluation image reveals across all classes.

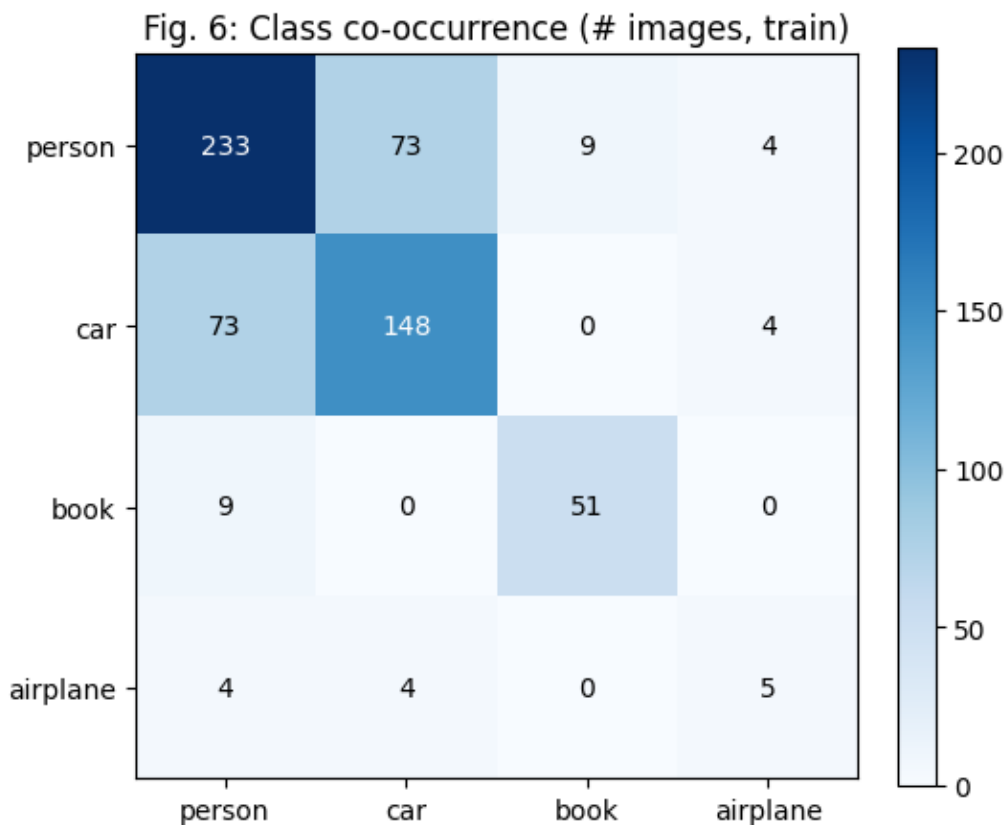


Figure 4: Class co-occurrence: training images containing each class pair

3. Methodology

Ground-truth annotations are instance-level RLE masks. We built a preprocessing tool decoding each instance via the COCO API and painting it onto a single-channel semantic mask, largest-area-first, so smaller/closer objects are not covered by larger ones beneath them. A sanity check confirmed every mask contained only the expected $\{0,1,2,3,4\}$ label set.

Images are resized to 256×256 (bilinear), masks use nearest-neighbour interpolation to avoid invalid fractional labels at boundaries. Training images are augmented with flips, small rotations, and brightness/contrast the main practical lever for exposing the model to more variety of the rare classes without new data, we visually verified augmented image-mask pairs stayed aligned.

Both models output 5 channels and are trained with cross-entropy weighted by log-smoothed inverse pixel frequency, chosen over raw inverse frequency because the latter’s extreme weighting (background: airplane = 760:1) destabilised training. Model 1 (U-Net, 7.77M parameters) is trained fully from scratch for 25 epochs. Model 2 (DeepLabV3-ResNet50, 42.0M

parameters) starts from COCO-pretrained weights with a replaced classifier head, fine-tuned for 15 epochs at a lower learning rate to preserve pretrained features. We track train/validation loss each epoch to monitor overfitting, checkpointing the best-mIoU epoch for final evaluation rather than trusting the last one.

4. Results and Discussion

DeepLabV3 outperformed U-Net on every class (Table 1, Fig. 5): mean IoU across target classes was 0.274 vs 0.085 roughly 3.2× higher.

Class	U-Net IoU	U-Net Dice	DeepLabV3 IoU	DeepLabV3 Dice
Background	0.830	0.907	0.912	0.954
Person	0.185	0.312	0.393	0.565
Car	0.153	0.266	0.445	0.616
Book	0.003	0.006	0.187	0.314
Airplan	0.000	0.000	0.069	0.129

Table 1: Per-class validation IoU/Dice

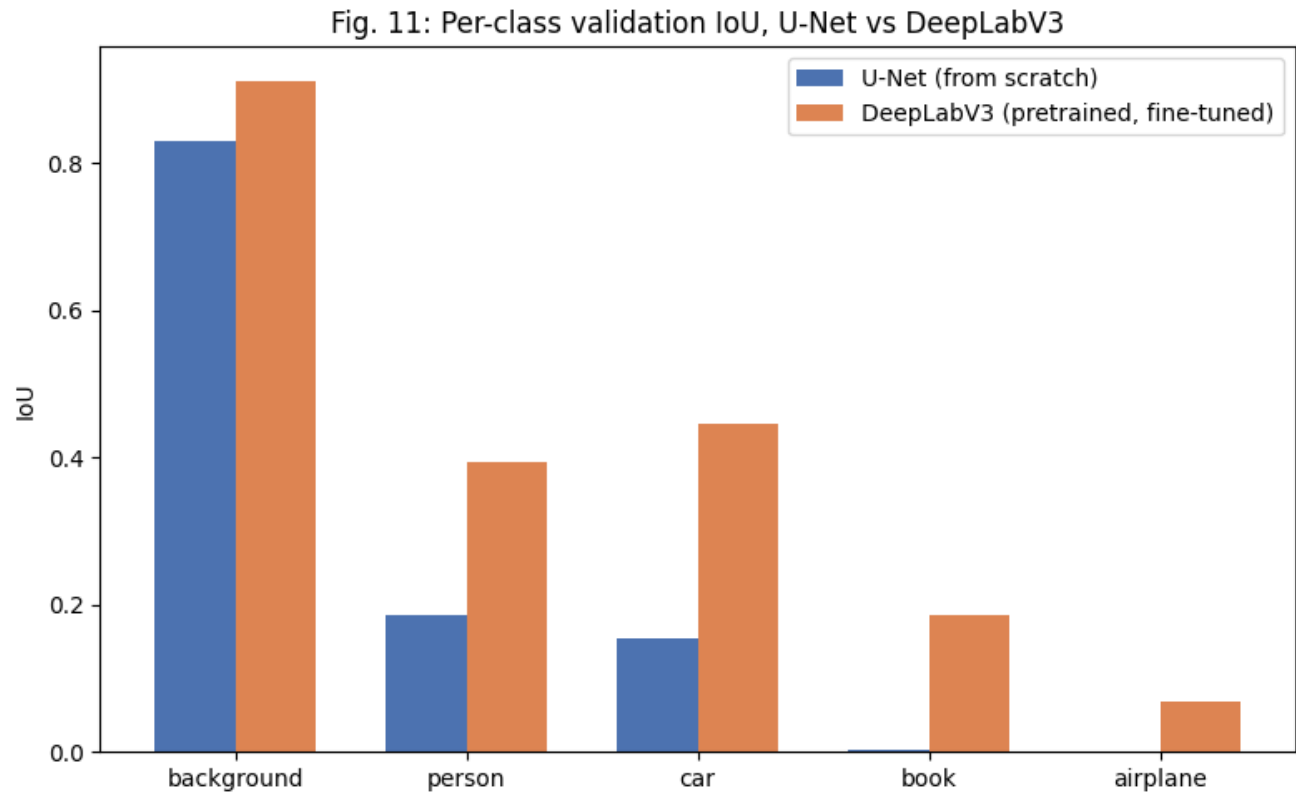


Figure 5: Per-class validation IoU, U-Net vs. DeepLabV3

The gap is largest exactly where our data is weakest: on airplane, U-Net scored 0.000 IoU, never learning to detect it from only 5 training images, while DeepLabV3 reached a still-low but non-zero 0.069 consistent with its COCO-pretrained backbone supplying transferable features our from-scratch model had no data to learn. Book was the second-hardest class for both (U-Net 0.003, DeepLabV3 0.187), which we attribute to scarcity compounded by the small instance sizes found in our EDA (Fig. 3). Person and car, the best-represented classes, were the strongest performers for both models. Pixel accuracy (U-Net 0.832, DeepLabV3 0.916) was markedly higher than target-class mIoU for both precisely the trap our brief warns against, since an always-background predictor would already score well here while being useless at the actual task.

Our worst-case analysis (Fig. 6) showed the U-Net’s hardest failures typically missed an object entirely rather than mislocalising it, reinforcing that data scarcity, not model capacity, is the dominant bottleneck.

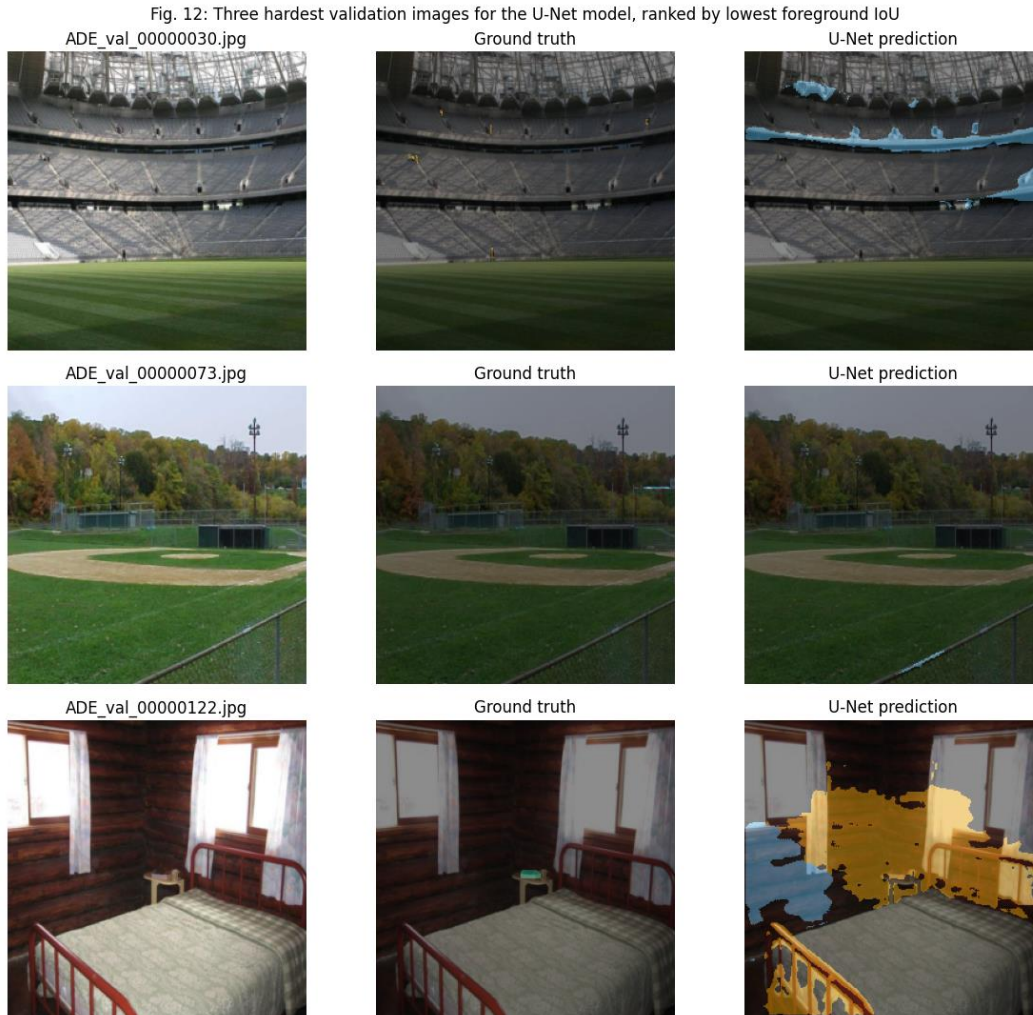


Figure 6: Three lowest-IoU validation images: input, ground truth, prediction

Qualitative evaluation on 3 unlabelled test images (Fig. 7) showed DeepLabV3 producing more coherent, contiguous regions than U-Net, whose predictions were noticeably fragmented consistent with the Dice gap (e.g. person: 0.312 vs 0.565).

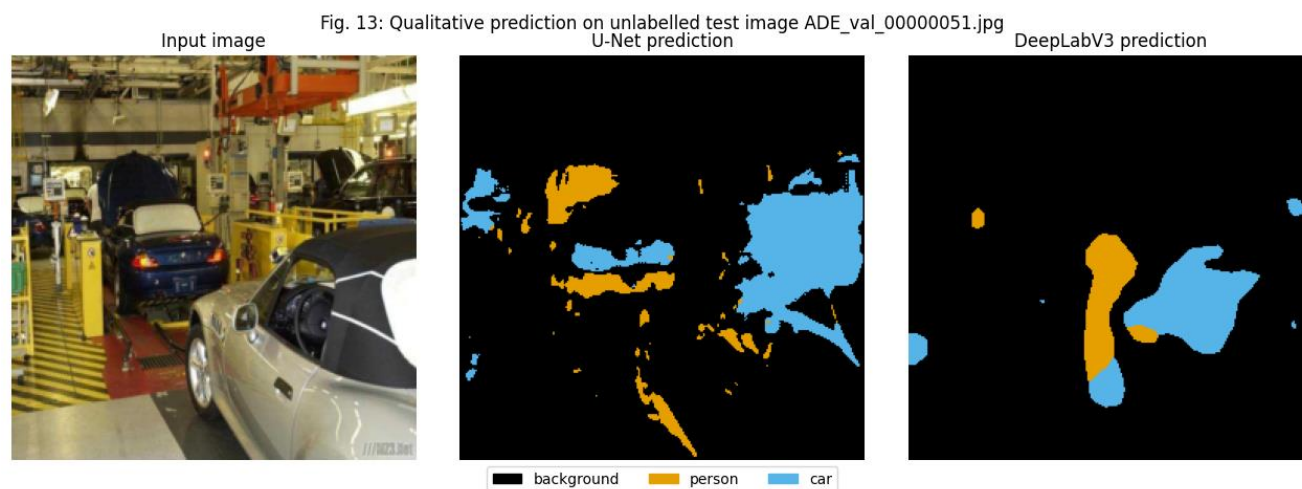


Figure 7a: Prediction on test image ADE_val_00000051.jpg

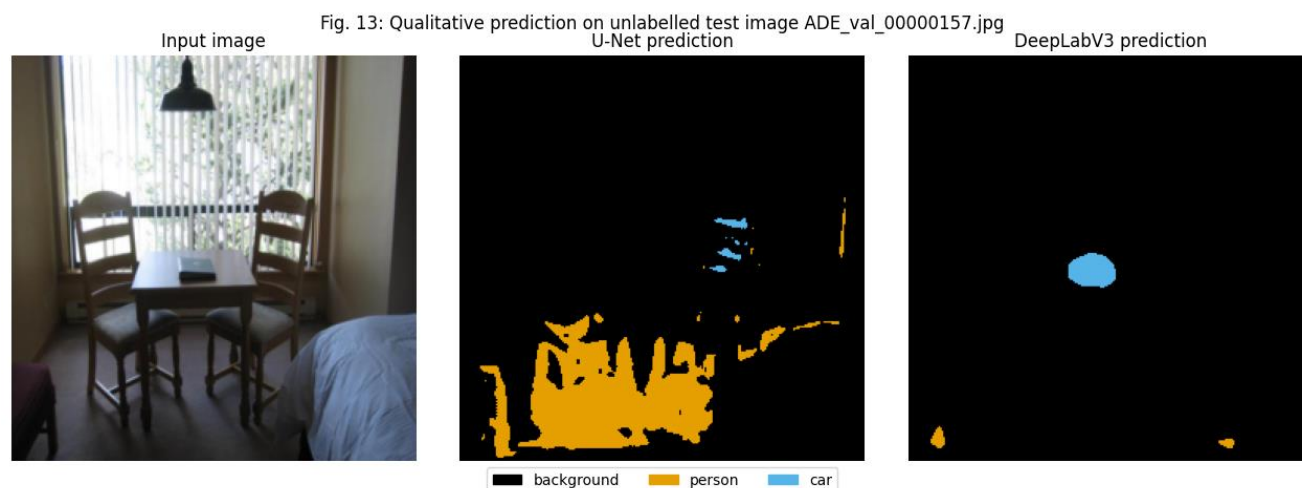


Figure 8: Prediction on test image ADE_val_00000157.jpg

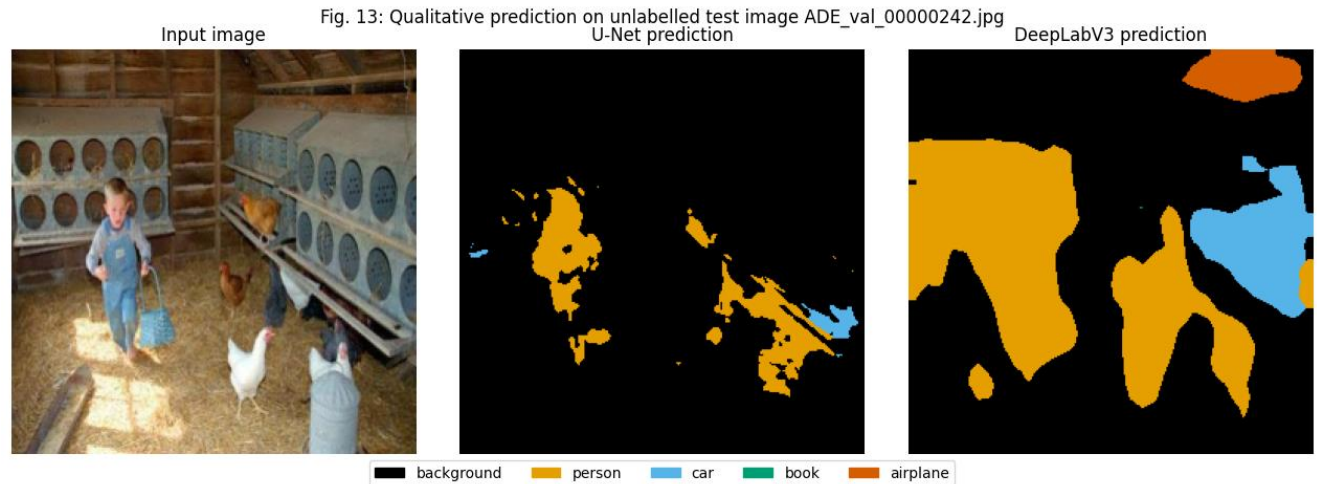


Figure 9: Prediction on test image ADE_val_00000242.jpg

Our absolute mIoU is far below published state-of-the-art on the full ADE20K benchmark (Zhou et al., 2017), this is expected, not directly comparable, given we used roughly 2% of that training data restricted to 4 classes. The clearest actionable improvement is acquiring or augmenting more airplane and book examples rather than further architecture tuning, since our results consistently point to data scarcity, not model capacity, as the limiting factor.

References

Chen, Liang-Chieh et al. “Rethinking Atrous Convolution for Semantic Image Segmentation.”

<https://arxiv.org/abs/1706.05587>

Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In: Navab, N., Hornegger, J., Wells, W., Frangi, A. (eds) Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. MICCAI 2015. Lecture Notes in Computer Science(), vol 9351. Springer, Cham. https://doi.org/10.1007/978-3-319-24574-4_28

Zhou, B., Zhao, H., Puig, X., Fidler, S., Barriuso, A., & Torralba, A. (2017). Scene Parsing through ADE20K Dataset. 2017 <https://arxiv.org/abs/1608.05442>

Appendix:

Semantic Segmentation on ADE20K (Person, Car, Book, Airplane)

Image Segmentation Assignment

Importing Libraries

"""

!pip install pycocotools -q

Commented out IPython magic to ensure Python compatibility.

import os

import random

from collections import defaultdict, Counter

import numpy as np

import matplotlib.pyplot as plt

import matplotlib.patches as mpatches

import skimage.io as io

from PIL import Image

import torch

import torch.nn as nn

import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader

import torchvision.transforms.functional as TF

from torchvision.models.segmentation import deeplabv3_resnet50,
DeepLabV3_ResNet50_Weights

```
from pycocotools.coco import COCO
```

```
# %matplotlib inline
```

```
SEED = 42
```

```
random.seed(SEED)
```

```
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
print('Using device:', device)
```

```
pip install squarify -q
```

```
"""### Mounting Dataset"""
```

```
from google.colab import drive
```

```
drive.mount("/content/drive")
```

```
DATA_ROOT =
```

```
"/content/drive/MyDrive/RMDS/RMDS_segmentation_dataset_ADE20K_350"
```

```
!find {DATA_ROOT} -name "__MACOSX" -exec rm -rf {} + 2>/dev/null
```

```
"""### loading training and validation paths"""
```

```
TRAIN_JSON = os.path.join(DATA_ROOT, 'instances_train.json')
VAL_JSON = os.path.join(DATA_ROOT, 'instances_val.json')
TRAIN_DIR = os.path.join(DATA_ROOT, 'train')
VAL_DIR = os.path.join(DATA_ROOT, 'val')
TEST_DIR = os.path.join(DATA_ROOT, 'test')
```

```
print('Train images:', len(os.listdir(TRAIN_DIR)))
print('Val images: ', len(os.listdir(VAL_DIR)))
print('Test images: ', len(os.listdir(TEST_DIR)))
```

```
"""### Loading The COCO API"""
```

```
coco_train = COCO(TRAIN_JSON)
coco_val = COCO(VAL_JSON)
```

```
category_IDs = coco_train.getCatIds()
categories = coco_train.loadCats(category_IDs)
```

```
names_cats = [c['name'] for c in categories]
print(len(names_cats), 'COCO categories:', ', '.join(names_cats))
```

```
def get_category_name(class_ID, categories):
    for c in categories:
        if c['id'] == class_ID:
            return c['name']
    return 'None'
```

```

# Sanity check, as in Tutorial 3
print('Category 4 is:', get_category_name(4, categories))
print('Category 9 is:', get_category_name(9, categories))
print('Category 40 is:', get_category_name(40, categories))
print('Category 59 is:', get_category_name(59, categories))

"""## Target Classes: Mapping COCO Category IDs to Our Semantic Labels

"""

target_classes = ['person', 'car', 'book', 'airplane']
target_classes_IDs = coco_train.getCatIds(catNms=target_classes)

CAT_ID_TO_LABEL = {cid: target_classes.index(get_category_name(cid, categories)) +
1
                    for cid in target_classes_IDs}
LABEL_TO_NAME = {0: 'background'}
LABEL_TO_NAME.update({i + 1: name for i, name in enumerate(target_classes)})
N_CLASSES = len(LABEL_TO_NAME)

print('CLASS MAPPING: COCO category_id to our semantic label')
for cls_name in target_classes:
    coco_id = target_classes_IDs[target_classes.index(cls_name)]
    our_label = CAT_ID_TO_LABEL[coco_id]
    print(f' COCO category_id {coco_id:>3} ("{cls_name}") to our label {our_label}')
print(f' {"(none - true background)":<38} to our label 0')
print()

```

```

print(' CAT_ID_TO_LABEL =', CAT_ID_TO_LABEL)
print(' LABEL_TO_NAME  =', LABEL_TO_NAME)
print(f' N_CLASSES = {N_CLASSES} (background + 4 target classes)')

"""## Exploratory Data Analysis (EDA)

### Dataset size per split
"""

print(f"Train folder: {len(os.listdir(TRAIN_DIR))} images, {len(coco_train.getImgIds())}
JSON entries")

print(f"Val folder:  {len(os.listdir(VAL_DIR))} images, {len(coco_val.getImgIds())} JSON
entries")

print(f"Test folder: {len(os.listdir(TEST_DIR))} images (unlabelled, no annotation file)")

print(f"Total annotations (all 100 categories) - train: {len(coco_train.anns)}, val:
{len(coco_val.anns)}")

train_filenames = {im['file_name'] for im in coco_train.imgs.values()}
val_filenames   = {im['file_name'] for im in coco_val.imgs.values()}
overlap = train_filenames & val_filenames

print(f"Filename overlap between train and val splits: {len(overlap)}")

assert len(overlap) == 0, "Data leakage detected: some images appear in both train and
val!"

print("No leakage detected - train and val splits are disjoint.")

"""### How each target class appear for image-level and instance-level"""

def target_class_stats(coco_obj, target_classes):

```

```

ids = coco_obj.getCatIds(catNms=target_classes)
img_counts, inst_counts = {}, {}
for name, cid in zip(target_classes, ids):
    img_counts[name] = len(coco_obj.getImgIds(catIds=[cid]))
    inst_counts[name] = len(coco_obj.getAnnIds(catIds=[cid]))
return img_counts, inst_counts

```

```

train_img_counts, train_inst_counts = target_class_stats(coco_train, target_classes)
val_img_counts, val_inst_counts = target_class_stats(coco_val, target_classes)

```

```

print(f'{"Class":<10}{"Train imgs":>12}{"Train inst.':>13}{"Val imgs":>10}{"Val inst.':>11}")
for name in target_classes:
    print(f'{"name":<10}{"train_img_counts[name]:>12}{"train_inst_counts[name]:>13}{"val_img_counts[name]:>10}{"val_inst_counts[name]:>11}")

```

```

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
x = np.arange(len(target_classes))
w = 0.35

```

```

axes[0].bar(x - w/2, [train_img_counts[n] for n in target_classes], w, label='Train',
color='#4C72B0')
axes[0].bar(x + w/2, [val_img_counts[n] for n in target_classes], w, label='Val',
color='#DD8452')
axes[0].set_xticks(x); axes[0].set_xticklabels(target_classes)
axes[0].set_ylabel('Number of images containing class')
axes[0].set_title('Fig. 1a: Image-level class frequency')
axes[0].legend()

```

```

axes[1].bar(x - w/2, [train_inst_counts[n] for n in target_classes], w, label='Train',
color='#4C72B0')

axes[1].bar(x + w/2, [val_inst_counts[n] for n in target_classes], w, label='Val',
color='#DD8452')

axes[1].set_xticks(x); axes[1].set_xticklabels(target_classes)

axes[1].set_yscale('log')

axes[1].set_ylabel('Number of instances (log scale)')

axes[1].set_title('Fig. 1b: Instance-level class frequency (log scale)')

axes[1].legend()


plt.tight_layout()

plt.savefig('fig1_class_frequency.png', dpi=120)

plt.show()

print("Fig. 1: Class frequency, computed via the COCO API's getImgIds/getAnnIds over
the 4 target")

print("classes. (a) Number of images containing at least one instance of each class.")

print("(b) Total instance count per class (log scale - 'airplane' is roughly 50-80x rarer
than 'person').")


"""### Image size distribution"""


train_imgs_info = coco_train.loadImgs(coco_train.getImgIds())

widths = [im['width'] for im in train_imgs_info]

heights = [im['height'] for im in train_imgs_info]

aspect_ratios = [w / h for w, h in zip(widths, heights)]


fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

axes[0].scatter(widths, heights, alpha=0.5, s=15, color='#4C72B0')

```

```

axes[0].set_xlabel('Width (px)'); axes[0].set_ylabel('Height (px)')
axes[0].set_title('Fig. 2a: Image resolution scatter (train)')

axes[1].hist(aspect_ratios, bins=30, color='#55A868', edgecolor='white')
axes[1].axvline(1.0, color='k', linestyle='--', linewidth=1, label='square (1:1)')
axes[1].set_xlabel('Aspect ratio (width / height)'); axes[1].set_ylabel('Count')
axes[1].set_title('Fig. 2b: Aspect ratio distribution (train)')
axes[1].legend()

plt.tight_layout()
plt.savefig('fig2_image_geometry.png', dpi=120)
plt.show()

print(f"Width range: {min(widths)}-{max(widths)} px, Height range: {min(heights)}-
{max(heights)} px")

print("Fig. 2: (a) Scatter of raw image resolutions in the training set - images are not a
fixed")

print("size, so batching requires resizing. (b) Aspect ratio distribution; most images are")

"""### Segmentation ground and encoding"""

print(f"Total annotations across all 100 categories - train: {len(coco_train.anns)}, val:
{len(coco_val.anns)}")

print(f"Annotations restricted to our 4 target classes - train:
{sum(train_inst_counts.values())}, "
      f"val: {sum(val_inst_counts.values())}")

example_ann =
coco_train.loadAnns(coco_train.getAnnIds(catIds=target_classes_IDs))[0]

print()

```

```
print('Example annotation dictionary keys:', list(example_ann.keys()))
print('Example segmentation field type:', type(example_ann['segmentation']))
print('Example segmentation field content:', example_ann['segmentation'] if
instance(example_ann['segmentation'], dict) else '(polygon list)')
```

```
##### Pixel-level class balance#####
```

```
def compute_pixel_balance(coco_obj, target_classes_IDs, cat_id_to_label, n_classes):
    img_ids = coco_obj.getImgIds()
    pixel_counts = Counter({i: 0 for i in range(n_classes)})
    for img_id in img_ids:
        info = coco_obj.loadImgs(img_id)[0]
        h, w = info['height'], info['width']
        mask = np.zeros((h, w), dtype=np.uint8)
        ann_ids = coco_obj.getAnnIds(imgIds=img_id, catIds=target_classes_IDs,
iscrowd=None)
        anns = coco_obj.loadAnns(ann_ids)
        anns = sorted(anns, key=lambda a: a['area'], reverse=True) # largest first, smaller
on top
        for a in anns:
            m = coco_obj.annToMask(a)
            mask[m > 0] = cat_id_to_label[a['category_id']]
        vals, counts = np.unique(mask, return_counts=True)
        for v, c in zip(vals, counts):
            pixel_counts[int(v)] += int(c)
    return pixel_counts
```

```

train_pixel_counts = compute_pixel_balance(coco_train, target_classes_IDs,
CAT_ID_TO_LABEL, N_CLASSES)

total_px = sum(train_pixel_counts.values())

print(f'{"Class":<10}{"Pixels":>14}{"% of all pixels":>18}')

for label in range(N_CLASSES):
    name = LABEL_TO_NAME[label]
    px = train_pixel_counts[label]
    print(f'{"name":<10}{"px":>14,}{"100*px/total_px:>17.3f}%'")

fig, ax = plt.subplots(figsize=(6,5))
labels_ = [LABEL_TO_NAME[i] for i in range(N_CLASSES)]
sizes = [train_pixel_counts[i] for i in range(N_CLASSES)]
colors_ = ['#B0B0B0', '#4C72B0', '#DD8452', '#55A868', '#C44E52']
ax.pie(sizes, labels=labels_, autopct='%1.2f%%', colors=colors_, startangle=90)
ax.set_title('Fig. 3: Pixel-level class balance (train split)')
plt.tight_layout()
plt.savefig('fig3_pixel_balance.png', dpi=120)
plt.show()

print("Fig. 3: Share of total image pixels belonging to each semantic class across the
training")

import squarify

treemap_labels = [f'{LABEL_TO_NAME[i]}\n{100*sizes[i]/sum(sizes):.2f}%' for i in
range(N_CLASSES)]

plt.figure(figsize=(9, 5))
squarify.plot(sizes=sizes, label=treemap_labels, color=colors_, alpha=0.85,

```

```

        text_kwargs={'fontsize': 10})
plt.axis('off')
plt.title('Fig. 4: Pixel share per class (train) - treemap view')
plt.tight_layout()
plt.savefig('fig4_pixel_balance_treemap.png', dpi=120)
plt.show()

print("Fig. 4: The same pixel-share data as Fig. 3, shown as a treemap. Background's
rectangle")

```

```

"""### Instance size distribution per class"""

```

```

def instance_area_fraction(coco_obj, target_classes, id_to_name):
    areas_by_class = defaultdict(list)
    for a in coco_obj.anns.values():
        if a['category_id'] in id_to_name:
            info = coco_obj.imgs[a['image_id']]
            frac = a['area'] / (info['width'] * info['height'])
            areas_by_class[id_to_name[a['category_id']]].append(frac * 100)
    return areas_by_class

```

```

id_to_name = {cid: get_category_name(cid, categories) for cid in target_classes_IDs}
areas_by_class = instance_area_fraction(coco_train, target_classes, id_to_name)

```

```

fig, ax = plt.subplots(figsize=(7, 5))
data_ = [areas_by_class[name] for name in target_classes]
colors_box = ['#4C72B0', '#DD8452', '#55A868', '#C44E52']
bp = ax.boxplot(data_, tick_labels=target_classes, showfliers=True, patch_artist=True)

```

```

for patch, c in zip(bp['boxes'], colors_box):
    patch.set_facecolor(c)
ax.set_ylabel('Instance area as % of image area')
ax.set_yscale('log')
ax.set_title('Fig. 4: Distribution of instance size per class (train, log scale)')
plt.tight_layout()
plt.savefig('fig4_instance_sizes.png', dpi=120)
plt.show()

print("Fig. 5: Box plot of individual object instance sizes (as a percentage of total image area)")

```

```

for name in target_classes:
    arr = np.array(areas_by_class[name])

    print(f'{name:<10}
n={len(arr):>4} median={np.median(arr):.3f}% max={arr.max():.2f}%")

```

```

"""### Class co-occurrence"""

```

```

def cooccurrence_matrix(coco_obj, target_classes, target_classes_IDs):
    img_ids = coco_obj.getImgIds()
    classes_per_img = defaultdict(set)
    for img_id in img_ids:
        ann_ids = coco_obj.getAnnIds(imgIds=img_id, catIds=target_classes_IDs,
iscrowd=None)
        for a in coco_obj.loadAnns(ann_ids):
            classes_per_img[img_id].add(get_category_name(a['category_id'], categories))
    mat = np.zeros((4, 4), dtype=int)
    for classes_present in classes_per_img.values():

```

```

    for i, ci in enumerate(target_classes):
        for j, cj in enumerate(target_classes):
            if ci in classes_present and cj in classes_present:
                mat[i, j] += 1
return mat

```

```

co_mat = cooccurrence_matrix(coco_train, target_classes, target_classes_IDs)
fig, ax = plt.subplots(figsize=(5.5, 5))
im = ax.imshow(co_mat, cmap='Blues')
ax.set_xticks(range(4)); ax.set_xticklabels(target_classes)
ax.set_yticks(range(4)); ax.set_yticklabels(target_classes)
for i in range(4):
    for j in range(4):
        ax.text(j, i, co_mat[i, j], ha='center', va='center',
                color='white' if co_mat[i, j] > co_mat.max() / 2 else 'black')
ax.set_title('Fig. 6: Class co-occurrence (# images, train)')
plt.colorbar(im, fraction=0.046)
plt.tight_layout()
plt.savefig('fig5_cooccurrence.png', dpi=120)
plt.show()
print("Fig. 6: Number of training images in which each pair of target classes co-occurs (diagonal)")

```

```

##### Qualitative check: original image, COCO-drawn annotations, and our decoded mask#####

```

```

example_img_id =
coco_train.getImgIds(catIds=coco_train.getCatIds(catNms=['person']))[10]

```

```
example_img_info = coco_train.loadImgs(example_img_id)[0]
example_image = io.imread(os.path.join(TRAIN_DIR, example_img_info['file_name']))
```

```
ann_ids = coco_train.getAnnIds(imgIds=example_img_id, catIds=target_classes_IDs,
iscrowd=None)
anns = coco_train.loadAnns(ann_ids)
```

```
plt.figure(figsize=(6, 4.5))
plt.imshow(example_image)
plt.axis('off')
coco_train.showAnns(anns, draw_bbox=False)
plt.title(f"Fig. 7: {example_img_info['file_name']} with COCO instance annotations
overlaid")
plt.tight_layout()
plt.savefig('fig7_coco_showanns.png', dpi=120, bbox_inches='tight')
plt.show()

print("Fig. 7: Training image with its target-class instance annotations drawn using the
COCO API's")
```

```
"""# Processing"""
```

```
def build_semantic_mask(coco_obj, image_id, target_classes_IDs, cat_id_to_label):
    info = coco_obj.loadImgs(image_id)[0]
    h, w = info['height'], info['width']
    mask = np.zeros((h, w), dtype=np.uint8)

    ann_ids = coco_obj.getAnnIds(imgIds=image_id, catIds=target_classes_IDs,
iscrowd=None)
```

```
anns = coco_obj.loadAnns(ann_ids)
anns = sorted(anns, key=lambda a: a['area'], reverse=True)
```

```
for a in anns:
    decoded = coco_obj.annToMask(a)
    label = cat_id_to_label[a['category_id']]
    mask[decoded > 0] = label
```

```
return mask, info['file_name']
```

```
def build_and_save_all_masks(coco_obj, out_dir, target_classes_IDs, cat_id_to_label):
    os.makedirs(out_dir, exist_ok=True)
    instance_counts = Counter()
    for img_id in coco_obj.getImgIds():
        mask, fname = build_semantic_mask(coco_obj, img_id, target_classes_IDs,
cat_id_to_label)
        for label in np.unique(mask):
            if label != 0:
                pass
            out_name = os.path.splitext(fname)[0] + '.png'
            Image.fromarray(mask).save(os.path.join(out_dir, out_name))
    return instance_counts
```

```
MASKS_TRAIN_DIR = os.path.join(DATA_ROOT, 'masks_train')
```

```
MASKS_VAL_DIR = os.path.join(DATA_ROOT, 'masks_val')
```

```
build_and_save_all_masks(coco_train, MASKS_TRAIN_DIR, target_classes_IDs,
CAT_ID_TO_LABEL)
```

```

build_and_save_all_masks(coco_val, MASKS_VAL_DIR, target_classes_IDs,
CAT_ID_TO_LABEL)

print('Masks written to:', MASKS_TRAIN_DIR, 'and', MASKS_VAL_DIR)
for label in range(N_CLASSES):
    print(f' pixel value {label} = {LABEL_TO_NAME[label]}')

sample_files = os.listdir(MASKS_TRAIN_DIR)[:20]
all_labels_seen = set()
for f in sample_files:
    m = np.array(Image.open(os.path.join(MASKS_TRAIN_DIR, f)))
    all_labels_seen.update(np.unique(m).tolist())

print('Labels seen across a sample of masks:', sorted(all_labels_seen))
assert all_labels_seen.issubset(set(LABEL_TO_NAME.keys())), 'Unexpected label
values found in masks!'

print('all mask values are within the expected {0,1,2,3,4} label set.')

mask, fname = build_semantic_mask(coco_train, example_img_id, target_classes_IDs,
CAT_ID_TO_LABEL)
CLASS_COLORS = np.array([
    [0, 0, 0],
    [230, 159, 0],
    [86, 180, 233],
    [0, 158, 115],
    [213, 94, 0],
])

```

```

overlay = CLASS_COLORS[mask]
blended = (0.55 * example_image[..., :3] + 0.45 * overlay).astype(np.uint8)

fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
axes[0].imshow(mask, cmap='tab10', vmin=0, vmax=4)
axes[0].set_title('Our semantic mask (labels 0-4)')
axes[0].axis('off')
axes[1].imshow(blended)
axes[1].set_title('Overlay on original image')
axes[1].axis('off')

present = sorted(set(np.unique(mask)) - {0})
handles = [mpatches.Patch(color=CLASS_COLORS[c]/255,
label=LABEL_TO_NAME[c]) for c in present]
if handles:
    fig.legend(handles=handles, loc='lower center', ncol=len(handles),
bbox_to_anchor=(0.5, -0.05))
plt.suptitle(f'Fig. 8: {fname} - our corrected preprocessing output')
plt.tight_layout()
plt.savefig('fig8_corrected_mask_example.png', dpi=120, bbox_inches='tight')
plt.show()

print(f'Fig. 8: Semantic mask produced by our corrected preprocessing tool for
'{fname}', matching the")

```

```

"""## PyTorch Dataset, Augmentation, and Class-Imbalance Handling

```

```

"""

```

```

class ADESegDataset(Dataset):

    def __init__(self, img_dir, mask_dir, file_list, img_size=256, augment=False):
        self.img_dir = img_dir
        self.mask_dir = mask_dir
        self.files = file_list
        self.img_size = img_size
        self.augment = augment
        self.mean = [0.485, 0.456, 0.406]
        self.std = [0.229, 0.224, 0.225]

    def __len__(self):
        return len(self.files)

    def __getitem__(self, idx):
        fname = self.files[idx]
        img_path = os.path.join(self.img_dir, fname)
        mask_path = os.path.join(self.mask_dir, os.path.splitext(fname)[0] + '.png')

        img = Image.open(img_path).convert('RGB')
        mask = Image.open(mask_path)

        img = img.resize((self.img_size, self.img_size), Image.BILINEAR)
        mask = mask.resize((self.img_size, self.img_size), Image.NEAREST)

        if self.augment:
            if random.random() > 0.5:
                img = TF.hflip(img)

```

```

        mask = TF.hflip(mask)
    if random.random() > 0.5:
        angle = random.uniform(-10, 10)
        img = TF.rotate(img, angle, interpolation=TF.InterpolationMode.BILINEAR)
        mask = TF.rotate(mask, angle, interpolation=TF.InterpolationMode.NEAREST)
    if random.random() > 0.5:
        img = TF.adjust_brightness(img, random.uniform(0.8, 1.2))
        img = TF.adjust_contrast(img, random.uniform(0.8, 1.2))

    img_t = TF.to_tensor(img)
    img_t = TF.normalize(img_t, self.mean, self.std)
    mask_t = torch.from_numpy(np.array(mask, dtype=np.int64))

    return img_t, mask_t

```

```

train_files = sorted(os.listdir(TRAIN_DIR))
val_files = sorted(os.listdir(VAL_DIR))

```

```

train_dataset = ADESegDataset(TRAIN_DIR, MASKS_TRAIN_DIR, train_files,
img_size=256, augment=True)

```

```

val_dataset =
ADESegDataset(VAL_DIR, MASKS_VAL_DIR, val_files, img_size=256,
augment=False)

```

```

BATCH_SIZE = 8

```

```

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True, num_workers=2)

```

```

val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
num_workers=2)

```

```
imgs, masks = next(iter(train_loader))
print('Image batch shape:', imgs.shape, imgs.dtype)
print('Mask batch shape: ', masks.shape, masks.dtype)
print('Unique labels present in this batch:', torch.unique(masks).tolist())
```

```
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
```

```
fig, axes = plt.subplots(2, 4, figsize=(14, 6.5))
for j in range(4):
    img_np = imgs[j].permute(1, 2, 0).numpy() * std + mean
    img_np = np.clip(img_np, 0, 1)
    mask_np = masks[j].numpy()
```

```
axes[0, j].imshow(img_np)
axes[0, j].axis('off')
```

```
overlay_color = CLASS_COLORS[mask_np] / 255.0
blended = 0.6 * img_np + 0.4 * overlay_color
axes[1, j].imshow(np.clip(blended, 0, 1))
axes[1, j].axis('off')
```

```
plt.suptitle('Fig. 9: Augmented training batch - images (top) and mask overlays (bottom)')
```

```
plt.tight_layout()
```

```
plt.savefig('fig9_1_augmented_batch_check.png', dpi=120, bbox_inches='tight')
```

```
plt.show()
```

```
print('Fig. 9: Four images from an augmented training batch (top row) with their  
corresponding')
```

```
"""### Class weighting to counter the imbalance quantified"""
```

```
def compute_class_weights(mask_dir, files, n_classes=N_CLASSES):
```

```
    counts = np.zeros(n_classes, dtype=np.int64)
```

```
    for fname in files:
```

```
        mpath = os.path.join(mask_dir, os.path.splitext(fname)[0] + '.png')
```

```
        m = np.array(Image.open(mpath))
```

```
        for c in range(n_classes):
```

```
            counts[c] += (m == c).sum()
```

```
    counts = np.maximum(counts, 1)
```

```
    freq = counts / counts.sum()
```

```
    weights = 1.0 / np.log(1.02 + freq)
```

```
    weights = weights / weights.sum() * n_classes
```

```
    return torch.tensor(weights, dtype=torch.float32), counts
```

```
class_weights, class_pixel_counts = compute_class_weights(MASKS_TRAIN_DIR,  
train_files)
```

```
print(f"{'Class':<10}{'Pixels':>14}{'Loss weight':>13}")
```

```
for i in range(N_CLASSES):
```

```
    print(f"{LABEL_TO_NAME[i]:<10}{class_pixel_counts[i]:>14,}{class_weights[i].item():  
>13.3f}")
```

```
class_weights = class_weights.to(device)
```

```
"""# Models
```

```
### Model 1: U-Net
```

```
"""
```

```
class DoubleConv(nn.Module):
```

```
    def __init__(self, in_ch, out_ch):
```

```
        super().__init__()
```

```
        self.block = nn.Sequential(
```

```
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
```

```
            nn.BatchNorm2d(out_ch),
```

```
            nn.ReLU(inplace=True),
```

```
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
```

```
            nn.BatchNorm2d(out_ch),
```

```
            nn.ReLU(inplace=True),
```

```
        )
```

```
    def forward(self, x):
```

```
        return self.block(x)
```

```
class UNet(nn.Module):
```

```
    def __init__(self, n_classes=5, base_ch=32):
```

```
        super().__init__()
```

```
        self.enc1 = DoubleConv(3, base_ch)
```

```
        self.enc2 = DoubleConv(base_ch, base_ch * 2)
```

```
        self.enc3 = DoubleConv(base_ch * 2, base_ch * 4)
```

```
        self.enc4 = DoubleConv(base_ch * 4, base_ch * 8)
```

```
        self.pool = nn.MaxPool2d(2)
```

```
self.bottleneck = DoubleConv(base_ch * 8, base_ch * 16)
```

```
self.up4 = nn.ConvTranspose2d(base_ch * 16, base_ch * 8, 2, stride=2)
```

```
self.dec4 = DoubleConv(base_ch * 16, base_ch * 8)
```

```
self.up3 = nn.ConvTranspose2d(base_ch * 8, base_ch * 4, 2, stride=2)
```

```
self.dec3 = DoubleConv(base_ch * 8, base_ch * 4)
```

```
self.up2 = nn.ConvTranspose2d(base_ch * 4, base_ch * 2, 2, stride=2)
```

```
self.dec2 = DoubleConv(base_ch * 4, base_ch * 2)
```

```
self.up1 = nn.ConvTranspose2d(base_ch * 2, base_ch, 2, stride=2)
```

```
self.dec1 = DoubleConv(base_ch * 2, base_ch)
```

```
self.out_conv = nn.Conv2d(base_ch, n_classes, 1)
```

```
def forward(self, x):
```

```
    e1 = self.enc1(x)
```

```
    e2 = self.enc2(self.pool(e1))
```

```
    e3 = self.enc3(self.pool(e2))
```

```
    e4 = self.enc4(self.pool(e3))
```

```
    b = self.bottleneck(self.pool(e4))
```

```
    d4 = self.up4(b); d4 = self.dec4(torch.cat([d4, e4], dim=1))
```

```
    d3 = self.up3(d4); d3 = self.dec3(torch.cat([d3, e3], dim=1))
```

```
    d2 = self.up2(d3); d2 = self.dec2(torch.cat([d2, e2], dim=1))
```

```
    d1 = self.up1(d2); d1 = self.dec1(torch.cat([d1, e1], dim=1))
```

```
    return self.out_conv(d1)
```

```

UNET_MODEL = UNet(n_classes=N_CLASSES, base_ch=32).to(device)
n_params_UNET = sum(p.numel() for p in UNET_MODEL.parameters())
print(f'U-Net parameter count: {n_params_UNET:,}')

with torch.no_grad():
    test_out = UNET_MODEL(imgs.to(device))
print('U-Net output shape:', test_out.shape, '(expected: [batch, 5, 256, 256])')
print()
print(f'The {N_CLASSES} output channels correspond exactly to the class mapping
from Section 1:')
for label in range(N_CLASSES):
    print(f' channel {label} = {LABEL_TO_NAME[label]}')

"""### Model 2: DeepLabV3 (ResNet-50, pretrained on COCO, fine-tuned)"""

def build_deeplab_model(n_classes=N_CLASSES, pretrained=True):
    weights = DeepLabV3_ResNet50_Weights.DEFAULT if pretrained else None
    model = deeplabv3_resnet50(weights=weights, aux_loss=True)
    model.classifier[4] = nn.Conv2d(256, n_classes, kernel_size=1)
    if model.aux_classifier is not None:
        model.aux_classifier[4] = nn.Conv2d(256, n_classes, kernel_size=1)
    return model

deeplab_model = build_deeplab_model(N_CLASSES, pretrained=True).to(device)
n_params_deeplab = sum(p.numel() for p in deeplab_model.parameters())
print(f'DeeplabV3 parameter count: {n_params_deeplab:,}')

```

```
with torch.no_grad():
    test_out = deeplab_model(imgs.to(device))
print('DeepLabV3 output keys:', list(test_out.keys()))
print('DeepLabV3 main output shape:', test_out['out'].shape)
```

```
"""## Training and Evaluation Metrics
```

```
"""
```

```
class ConfusionMeter:
```

```
    def __init__(self, n_classes=N_CLASSES):
        self.n = n_classes
        self.mat = torch.zeros(n_classes, n_classes, dtype=torch.long)
```

```
    def update(self, pred, target):
        k = (target * self.n + pred).reshape(-1).cpu()
        self.mat += torch.bincount(k, minlength=self.n ** 2).reshape(self.n, self.n)
```

```
    def scores(self):
        m = self.mat.float()
        tp = m.diag()
        fp = m.sum(0) - tp
        fn = m.sum(1) - tp
        iou = (tp / (tp + fp + fn).clamp(min=1)).numpy()
        dice = (2 * tp / (2 * tp + fp + fn).clamp(min=1)).numpy()
        pixel_acc = (tp.sum() / m.sum()).item()
```

```
return iou, dice, pixel_acc
```

```
@torch.no_grad()
```

```
def evaluate(model, loader, n_classes=N_CLASSES, is_deeplab=False):
```

```
    model.eval()
```

```
    meter = ConfusionMeter(n_classes)
```

```
    for imgs_b, masks_b in loader:
```

```
        imgs_b = imgs_b.to(device)
```

```
        out = model(imgs_b)
```

```
        logits = out["out"] if is_deeplab else out
```

```
        preds = torch.argmax(logits, dim=1).cpu()
```

```
        meter.update(preds, masks_b)
```

```
    return meter.scores()
```

```
"""### Training loop"""
```

```
def train_model(model, train_loader, val_loader, n_epochs, lr, is_deeplab=False,  
model_name='model'):
```

```
    criterion = nn.CrossEntropyLoss(weight=class_weights)
```

```
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

```
    history = {'train_loss': [], 'val_loss': [], 'val_miou': []}
```

```
    best_miou = -1.0
```

```
    ckpt_path = f'/content/{model_name}_best.pt'
```

```
    for epoch in range(n_epochs):
```

```
        model.train()
```

```
        running_loss = 0.0
```

```

for imgs_b, masks_b in train_loader:
    imgs_b, masks_b = imgs_b.to(device), masks_b.to(device)
    optimizer.zero_grad()
    out = model(imgs_b)
    if is_deeplab:
        loss = criterion(out['out'], masks_b) + 0.4 * criterion(out['aux'], masks_b)
    else:
        loss = criterion(out, masks_b)
    loss.backward()
    optimizer.step()
    running_loss += loss.item() * imgs_b.size(0)

epoch_loss = running_loss / len(train_loader.dataset)

model.eval()
running_val_loss = 0.0
meter = ConfusionMeter(N_CLASSES)
with torch.no_grad():
    for imgs_b, masks_b in val_loader:
        imgs_b, masks_b = imgs_b.to(device), masks_b.to(device)
        out = model(imgs_b)
        logits = out['out'] if is_deeplab else out
        if is_deeplab:
            vloss = criterion(out['out'], masks_b) + 0.4 * criterion(out['aux'], masks_b)
        else:
            vloss = criterion(logits, masks_b)
        running_val_loss += vloss.item() * imgs_b.size(0)

```

```

        preds = torch.argmax(logits, dim=1).cpu()
        meter.update(preds, masks_b.cpu())

    epoch_val_loss = running_val_loss / len(val_loader.dataset)
    iou_per_class, _, _ = meter.scores()
    val_miou = np.nanmean(iou_per_class[1:])

    history['train_loss'].append(epoch_loss)
    history['val_loss'].append(epoch_val_loss)
    history['val_miou'].append(val_miou)

    improved = "
    if val_miou > best_miou:
        best_miou = val_miou
        torch.save(model.state_dict(), ckpt_path)
        improved = ' (new best, checkpoint saved)'

    print(f'[{model_name}] epoch {epoch+1}/{n_epochs} train_loss={epoch_loss:.4f} '
          f'val_loss={epoch_val_loss:.4f} val_mIoU(target
classes)={val_miou:.4f}{improved}')

    model.load_state_dict(torch.load(ckpt_path))

    print(f'[{model_name}] training complete. Best val mIoU = {best_miou:.4f} (checkpoint
reloaded).')

    return history

N_EPOCHS = 25

LR = 1e-3

```

```
print('Training U-Net')

UNET_HISTORY = train_model(unet_model, train_loader, val_loader, N_EPOCHS, LR,
                           is_deeplab=False, model_name='unet')
```

```
N_EPOCHS_DEEPLAB = 15
```

```
LR_DEEPLAB = 5e-4
```

```
print('Fine-tuning DeepLabV3')
```

```
deeplab_history = train_model(deeplab_model, train_loader, val_loader,
                              N_EPOCHS_DEEPLAB, LR_DEEPLAB,
                              is_deeplab=True, model_name='deeplab')
```

```
"""### Training curves"""
```

```
fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
```

```
axes[0].plot(unet_history['train_loss'], label='train loss', color='#4C72B0')
axes[0].plot(unet_history['val_loss'], label='val loss', color='#C44E52')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Weighted CE loss')
axes[0].set_title('Fig. 10a: U-Net train vs. val loss'); axes[0].legend()
```

```
axes[1].plot(deeplab_history['train_loss'], label='train loss', color='#4C72B0')
axes[1].plot(deeplab_history['val_loss'], label='val loss', color='#C44E52')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Weighted CE loss (+ 0.4x aux)')
axes[1].set_title('Fig. 10b: DeepLabV3 train vs. val loss'); axes[1].legend()
```

```
axes[2].plot(unet_history['val_miou'], label='U-Net', color='#4C72B0')
```

```

axes[2].plot(range(len(deeplab_history['val_miou'])), deeplab_history['val_miou'],
label='DeepLabV3', color='#DD8452')

axes[2].set_xlabel('Epoch'); axes[2].set_ylabel('Validation mIoU (target classes)')
axes[2].set_title('Fig. 10c: Validation mIoU over training'); axes[2].legend()

plt.tight_layout()

plt.savefig('fig10_training_curves.png', dpi=120)

plt.show()

print('Fig. 10: (a,b) Training vs. validation loss per model,')
print(' used to check for overfitting')
print('(c) Validation mIoU (averaged over person/car/book/airplane, excluding')
print('background) across epochs, comparing the from-scratch U-Net against the fine-
tuned DeepLabV3.')

```

```

"""### Final quantitative comparison on the validation set"""

```

```

UNET_IoU, UNET_dice, UNET_pixel_acc = evaluate(unet_model, val_loader,
is_deeplab=False)

deeplab_IoU, deeplab_dice, deeplab_pixel_acc = evaluate(deeplab_model, val_loader,
is_deeplab=True)

coco_id_for_label = {0: '-'}

for cid, lbl in CAT_ID_TO_LABEL.items():
    coco_id_for_label[lbl] = cid

print(f'{"Label":<7}{"COCO id":<9}{"Class":<12}{"U-Net IoU":>11}{"U-Net Dice":>12}{"DeepLab IoU":>13}{"DeepLab Dice":>14}{"')

for i in range(N_CLASSES):
    print(f'{"i":<7}{"str(coco_id_for_label[i]):<9}{"LABEL_TO_NAME[i]:<12}{"')

```

```
f'{unet_iou[i]:>11.3f}{unet_dice[i]:>12.3f}{deeplab_iou[i]:>13.3f}{deeplab_dice[i]:>14.3f}')
```

```
unet_miou = np.nanmean(unet_iou[1:])
```

```
deeplab_miou = np.nanmean(deeplab_iou[1:])
```

```
print()
```

```
print(f'Mean IoU across target classes (excl. background): U-
```

```
Net={unet_miou:.3f} DeepLabV3={deeplab_miou:.3f}')
```

```
print(f'Overall pixel accuracy (all 5 classes incl. background): U-
```

```
Net={unet_pixel_acc:.3f} DeepLabV3={deeplab_pixel_acc:.3f}')
```

```
fig, ax = plt.subplots(figsize=(8,5))
```

```
x = np.arange(N_CLASSES)
```

```
w = 0.35
```

```
ax.bar(x - w/2, unet_iou, w, label='U-Net (from scratch)', color='#4C72B0')
```

```
ax.bar(x + w/2, deeplab_iou, w, label='DeepLabV3 (pretrained, fine-tuned)',  
color='#DD8452')
```

```
ax.set_xticks(x); ax.set_xticklabels([LABEL_TO_NAME[i] for i in range(N_CLASSES)])
```

```
ax.set_ylabel('IoU')
```

```
ax.set_title('Fig. 11: Per-class validation IoU, U-Net vs DeepLabV3')
```

```
ax.legend()
```

```
plt.tight_layout()
```

```
plt.savefig('fig11_iou_comparison.png', dpi=120)
```

```
plt.show()
```

```
print('Fig. 11: Per-class Intersection-over-Union on the held-out validation split,  
comparing')
```

```
@torch.no_grad()
```

```
def predict_full_mask(model, img_path, img_size=256, is_deeplab=False):
```

```

img = Image.open(img_path).convert('RGB')
img_resized = img.resize((img_size, img_size), Image.BILINEAR)
img_t = TF.to_tensor(img_resized)
img_t = TF.normalize(img_t, [0.485, 0.456, 0.406], [0.229, 0.224,
0.225]).unsqueeze(0).to(device)

out = model(img_t)
logits = out['out'] if is_deeplab else out
pred = torch.argmax(logits, dim=1).squeeze(0).cpu().numpy()
return pred, np.array(img_resized)

def foreground_iou(pred, gt):
    inter = ((pred == gt) & (gt > 0)).sum()
    union = ((pred > 0) | (gt > 0)).sum()
    return inter / max(union, 1)

rows = []
for fname in val_files:
    mask_path = os.path.join(MASKS_VAL_DIR, os.path.splitext(fname)[0] + '.png')
    gt_full = np.array(Image.open(mask_path))
    if gt_full.max() == 0:
        continue
    gt_resized = np.array(Image.fromarray(gt_full).resize((256, 256), Image.NEAREST))
    pred, _ = predict_full_mask(unet_model, os.path.join(VAL_DIR, fname),
is_deeplab=False)
    score = foreground_iou(pred, gt_resized)
    rows.append((fname, score))

rows.sort(key=lambda r: r[1])

```

```

worst_files = [r[0] for r in rows[:3]]
print('Three hardest validation images for the U-Net model (lowest foreground IoU):')
for fname, score in rows[:3]:
    print(f' {fname}: foreground IoU = {score:.4f}')

fig, axes = plt.subplots(3, 3, figsize=(12, 11))
for i, fname in enumerate(worst_files):
    img_path = os.path.join(VAL_DIR, fname)
    mask_path = os.path.join(MASKS_VAL_DIR, os.path.splitext(fname)[0] + '.png')
    gt_full = np.array(Image.open(mask_path))
    gt_resized = np.array(Image.fromarray(gt_full).resize((256, 256), Image.NEAREST))
    pred, img_resized = predict_full_mask(unet_model, img_path, is_deeplab=False)

    gt_overlay = (0.55 * img_resized + 0.45 *
CLASS_COLORS[gt_resized]).astype(np.uint8)

    pred_overlay = (0.55 * img_resized + 0.45 *
CLASS_COLORS[pred]).astype(np.uint8)

    axes[i, 0].imshow(img_resized); axes[i, 0].set_title(fname); axes[i, 0].axis('off')
    axes[i, 1].imshow(gt_overlay); axes[i, 1].set_title('Ground truth'); axes[i, 1].axis('off')
    axes[i, 2].imshow(pred_overlay); axes[i, 2].set_title('U-Net prediction'); axes[i,
2].axis('off')

plt.suptitle('Fig. 12: Three hardest validation images for the U-Net model, ranked by
lowest foreground IoU')

plt.tight_layout()

plt.savefig('fig12_worst_cases.png', dpi=120, bbox_inches='tight')

plt.show()

```

```
print('Fig. 12: The three validation images where the U-Net model scored lowest on foreground IoU')
```

```
"""## Qualitative Evaluation on 3 Unlabelled Test Images
```

```
"""
```

```
test_files = sorted(os.listdir(TEST_DIR))
```

```
print(f'{len(test_files)} unlabelled test images available.')
```

```
print(test_files[:10], '...')
```

```
chosen_test_files = test_files[:3]
```

```
print('Chosen for evaluation:', chosen_test_files)
```

```
def preprocess_for_inference(img_path, img_size=256):
```

```
    img = Image.open(img_path).convert('RGB')
```

```
    img_resized = img.resize((img_size, img_size), Image.BILINEAR)
```

```
    img_t = TF.to_tensor(img_resized)
```

```
    img_t = TF.normalize(img_t, [0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
```

```
    return img_t.unsqueeze(0), np.array(img_resized)
```

```
@torch.no_grad()
```

```
def predict_mask(model, img_tensor, is_deeplab=False):
```

```
    model.eval()
```

```
    img_tensor = img_tensor.to(device)
```

```
    out = model(img_tensor)
```

```
    logits = out['out'] if is_deeplab else out
```

```

pred = torch.argmax(logits, dim=1).squeeze(0).cpu().numpy()

return pred

for fname in chosen_test_files:

    img_path = os.path.join(TEST_DIR, fname)

    img_tensor, img_resized = preprocess_for_inference(img_path)

    unet_pred = predict_mask(unet_model, img_tensor, is_deeplab=False)
    deeplab_pred = predict_mask(deeplab_model, img_tensor, is_deeplab=True)

    fig, axes = plt.subplots(1, 3, figsize=(13, 4.5))

    axes[0].imshow(img_resized); axes[0].set_title('Input image'); axes[0].axis('off')

    axes[1].imshow(CLASS_COLORS[unet_pred]); axes[1].set_title('U-Net prediction');
    axes[1].axis('off')

    axes[2].imshow(CLASS_COLORS[deeplab_pred]); axes[2].set_title('DeepLabV3
prediction'); axes[2].axis('off')

    present_classes = sorted(set(np.unique(unet_pred)) | set(np.unique(deeplab_pred)))

    handles = [mpatches.Patch(color=CLASS_COLORS[c]/255,
label=LABEL_TO_NAME[c]) for c in present_classes]

    fig.legend(handles=handles, loc='lower center', ncol=len(handles),
bbox_to_anchor=(0.5, -0.05))

    plt.suptitle(f'Fig. 13: Qualitative prediction on unlabelled test image {fname}')

    plt.tight_layout()

    plt.savefig(f'fig11_test_{fname}.png', dpi=120, bbox_inches='tight')

    plt.show()

    print(f'Fig. 13: Model predictions on unlabelled test image '{fname}' (no ground truth
available;")

```

```

class_example_images = {}
for cname, cid in zip(target_classes, target_classes_IDs):
    ann_ids = coco_val.getAnnIds(catIds=[cid])
    anns = coco_val.loadAnns(ann_ids)
    best_ann = max(anns, key=lambda a: a['area'])
    info = coco_val.loadImgs(best_ann['image_id'])[0]
    class_example_images[cname] = {
        'image_id': best_ann['image_id'],
        'file_name': info['file_name'],
        'area': best_ann['area'],
    }

print('Chosen example image per target class:')
for cname, info in class_example_images.items():
    print(f" {cname:<10} -> {info['file_name']} (instance area = {info['area']} px)")

@torch.no_grad()
def predict_on_val_image(model, image_id, coco_obj, img_dir, img_size=256,
is_deeplab=False):
    info = coco_obj.loadImgs(image_id)[0]
    img_path = os.path.join(img_dir, info['file_name'])
    img = Image.open(img_path).convert('RGB')
    img_resized = img.resize((img_size, img_size), Image.BILINEAR)

    img_t = TF.to_tensor(img_resized)
    img_t = TF.normalize(img_t, [0.485, 0.456, 0.406], [0.229, 0.224,
0.225]).unsqueeze(0).to(device)

```

```

out = model(img_t)

logits = out['out'] if is_deeplab else out

pred = torch.argmax(logits, dim=1).squeeze(0).cpu().numpy()

gt_mask, _ = build_semantic_mask(coco_obj, image_id, target_classes_IDs,
CAT_ID_TO_LABEL)

gt_resized = np.array(Image.fromarray(gt_mask).resize((img_size, img_size),
Image.NEAREST))

return np.array(img_resized), gt_resized, pred

results_per_class = {}
for cname, info in class_example_images.items():
    img_np, gt_np, unet_pred = predict_on_val_image(
        unet_model, info['image_id'], coco_val, VAL_DIR, is_deeplab=False)
    _, _, deeplab_pred = predict_on_val_image(
        deeplab_model, info['image_id'], coco_val, VAL_DIR, is_deeplab=True)
    results_per_class[cname] = (img_np, gt_np, unet_pred, deeplab_pred)

print('Predictions computed for all 4 target-class example images.')

fig, axes = plt.subplots(4, 4, figsize=(16, 15))
col_titles = ['Input image', 'Ground truth', 'U-Net prediction', 'DeepLabV3 prediction']

for row, cname in enumerate(target_classes):
    img_np, gt_np, unet_pred, deeplab_pred = results_per_class[cname]

    gt_overlay = (0.55 * img_np + 0.45 * CLASS_COLORS[gt_np]).astype(np.uint8)

```

```

    unet_overlay = (0.55 * img_np + 0.45 *
CLASS_COLORS[unet_pred]).astype(np.uint8)

    deeplab_overlay = (0.55 * img_np + 0.45 *
CLASS_COLORS[deeplab_pred]).astype(np.uint8)

panels = [img_np, gt_overlay, unet_overlay, deeplab_overlay]
for col, panel in enumerate(panels):
    axes[row, col].imshow(panel)
    axes[row, col].axis('off')
    if row == 0:
        axes[row, col].set_title(col_titles[col])
axes[row, 0].text(-0.15, 0.5, cname, transform=axes[row, 0].transAxes,
                 fontsize=13, fontweight='bold', rotation=90,
                 va='center', ha='center')

handles = [mpatches.Patch(color=CLASS_COLORS[i]/255, label=LABEL_TO_NAME[i])
for i in range(N_CLASSES)]
fig.legend(handles=handles, loc='lower center', ncol=N_CLASSES,
bbox_to_anchor=(0.5, -0.02))

plt.suptitle('Fig. 14: Per-class visual check - person, car, book, airplane', fontsize=14)
plt.tight_layout()

plt.savefig('fig14_per_class_visual_check.png', dpi=120, bbox_inches='tight')
plt.show()

print("Fig. 14: One validation example per target class (row), each with its ground-truth
mask and")

```